Soluciones: práctica de gestión de procesos (sesión 7)

Nov 3, 2026 · @Angela

Solución de la práctica de la sesión 7: el script `gestion_procesos.sh`, que se entrega en `bash/sesion7_procesos`. El script está probado en Ubuntu, como usuario normal y como root, y los ejemplos de ejecución son salidas reales de esas pruebas.

Script gestion_procesos.sh

```
#!/bin/bash
# gestion_procesos.sh - Menú para consultar y gestionar los procesos del
sistema

# Fichero donde se registran los cambios de prioridad y los procesos
finalizados
log=~/gestion_procesos.log

# Escribe una línea en el registro con el formato: fecha hora - acción - PID
- comando
# Parámetros: $1 acción, $2 PID, $3 comando
registrar() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') - $1 - $2 - $3" >> "$log"
}

# Devuelve 0 si el parámetro es un número y existe un proceso con ese PID; 1
si no
validar_pid() {
    if [[ $1 =~ ^[0-9]+$ ]] && ps -p "$1" > /dev/null; then
        return 0
    fi
    return 1
}

# Opción 1: muestra los procesos de un usuario, si el usuario existe
listar_usuario() {
    read -p "Usuario: " usuario
    if id "$usuario" &> /dev/null; then
        ps -u "$usuario" -o pid,pcpu,pmem,comm
    else
        echo "Error: el usuario $usuario no existe"
        return 1
```

```
    fi
}

# Opción 2: busca los procesos con el nombre indicado
buscar_proceso() {
    read -p "Nombre del proceso: " nombre
    if [[ -z $nombre ]]; then
        echo "Error: no has escrito ningún nombre"
        return 1
    fi
    # ps -C selecciona los procesos por nombre de comando y devuelve 1 si no
    # hay ninguno; se comprueba antes porque, aunque no haya, muestra la
cabecera
    if ps -C "$nombre" > /dev/null; then
        ps -C "$nombre" -o pid,user,pcpu,comm
    else
        echo "No hay ningún proceso que coincida con $nombre"
    fi
}

# Opción 3: muestra los N procesos que más CPU consumen (por defecto, 5)
# Parámetro: $1 número de procesos
top_cpu() {
    n=$1
    if [[ -z $n ]]; then
        n=5
    fi
    if [[ ! $n =~ ^[0-9]+$ ]]; then
        echo "Error: $n no es un número"
        return 1
    fi
    if [[ $n -eq 0 ]]; then
        echo "Error: el número debe ser mayor que 0"
        return 1
    fi
    # head muestra n + 1 líneas: la cabecera y los n procesos
    ps -eo pid,user,pcpu,comm --sort=-pcpu | head -n $((n + 1))
}

# Opción 4: cambia el valor nice de un proceso con renice
cambiar_prioridad() {
    read -p "PID: " pid
    if ! validar_pid "$pid"; then
        echo "Error: $pid no es un proceso válido"
        return 1
    fi
```

```
    read -p "Nuevo valor nice: " valor
    if [[ ! $valor =~ ^-?[0-9]+$ ]]; then
        echo "Error: $valor no es un número"
        return 1
    fi
    # Solo root puede poner valores negativos (más prioridad)
    if [[ $EUID -eq 0 ]]; then
        minimo=-20
    else
        minimo=0
    fi
    if [[ $valor -lt $minimo || $valor -gt 19 ]]; then
        echo "Error: el valor debe estar entre $minimo y 19"
        return 1
    fi
    comando=$(ps -p "$pid" -o comm=)
    if renice -n "$valor" -p "$pid" > /dev/null; then
        echo "Prioridad de $pid ($comando) cambiada a $valor"
        registrar "renice $valor" "$pid" "$comando"
    else
        echo "Error: no se ha podido cambiar la prioridad de $pid"
        return 1
    fi
}

# Opción 5: finaliza un proceso con SIGTERM y, si no termina, con SIGKILL
finalizar_proceso() {
    read -p "PID: " pid
    if ! validar_pid "$pid"; then
        echo "Error: $pid no es un proceso válido"
        return 1
    fi
    comando=$(ps -p "$pid" -o comm=)
    read -p "¿Finalizar $pid ($comando)? (s/n): " respuesta
    if [[ $respuesta != "s" ]]; then
        echo "Operación cancelada"
        return 0
    fi
    if ! kill -SIGTERM "$pid"; then
        echo "Error: no se ha podido enviar la señal a $pid"
        return 1
    fi
    # Se da tiempo al proceso para terminar de forma ordenada
    sleep 3
    if ps -p "$pid" > /dev/null; then
        kill -SIGKILL "$pid"
```

```
        echo "El proceso $pid no respondía: enviada SIGKILL"
        registrar "SIGKILL" "$pid" "$comando"
    else
        echo "Proceso $pid finalizado con SIGTERM"
        registrar "SIGTERM" "$pid" "$comando"
    fi
}

# Muestra las opciones del menú
menu() {
    echo "===== GESTIÓN DE PROCESOS ====="
    echo "1) Listar los procesos de un usuario"
    echo "2) Buscar un proceso por nombre"
    echo "3) Mostrar los N procesos que más CPU consumen"
    echo "4) Cambiar la prioridad de un proceso"
    echo "5) Finalizar un proceso"
    echo "6) Salir"
}

# Programa principal: muestra el menú hasta que se elige la opción 6
opcion=0
while [[ $opcion != "6" ]]; do
    menu
    read -p "Opción: " opcion
    case $opcion in
        1) listar_usuario ;;
        2) buscar_proceso ;;
        3) read -p "N (Intro = 5): " n
           top_cpu "$n" ;;
        4) cambiar_prioridad ;;
        5) finalizar_proceso ;;
        6) echo "Hasta luego" ;;
        *) echo "Opción no válida" ;;
    esac
    echo
done
```

Ejemplos de ejecución

Ejecutado como el usuario `alumno` (sin el menú):

```
Opción: 1
Usuario: alumno
  PID %CPU %MEM COMMAND
  238  0.0  0.0 sleep
```

```
  241  0.0  0.0 sleep
  252  0.0  0.0 gestion_proceso
  254  0.0  0.0 ps

Opción: 1
Usuario: pepito
Error: el usuario pepito no existe

Opción: 2
Nombre del proceso: sleep
  PID USER     %CPU COMMAND
  238 alumno    0.0 sleep
  241 alumno    0.0 sleep

Opción: 2
Nombre del proceso: firefox
No hay ningún proceso que coincida con firefox

Opción: 3
N (Intro = 5): 2
  PID USER     %CPU COMMAND
  337 alumno   99.2 backup.sh
  333 alumno   98.7 backup.sh

Opción: 3
N (Intro = 5): abc
Error: abc no es un número

Opción: 4
PID: 238
Nuevo valor nice: -5
Error: el valor debe estar entre 0 y 19

Opción: 4
PID: 238
Nuevo valor nice: 10
Prioridad de 238 (sleep) cambiada a 10

Opción: 4
PID: abc
Error: abc no es un proceso válido

Opción: 5
PID: 238
¿Finalizar 238 (sleep)? (s/n): n
Operación cancelada
```

```
Opción: 5
PID: 238
¿Finalizar 238 (sleep)? (s/n): s
Proceso 238 finalizado con SIGTERM

Opción: 5
PID: 241
¿Finalizar 241 (sleep)? (s/n): s
El proceso 241 no respondía: enviada SIGKILL

Opción: 7
Opción no válida

Opción: 6
Hasta luego
```

El proceso 241 se lanzó ignorando `SIGTERM`, para comprobar que el script recurre a `SIGKILL`.

```
$ cat ~/gestion_procesos.log
2026-11-03 11:15:48 - renice 10 - 238 - sleep
2026-11-03 11:15:51 - SIGTERM - 238 - sleep
2026-11-03 11:16:00 - renice 10 - 287 - sleep
2026-11-03 11:16:03 - SIGKILL - 241 - sleep
```

Ejecutado como root, se admiten valores negativos:

```
Opción: 4
PID: 287
Nuevo valor nice: -5
Prioridad de 287 (sleep) cambiada a -5

Opción: 4
PID: 287
Nuevo valor nice: 25
Error: el valor debe estar entre -20 y 19
```

Un usuario normal no puede bajar el nice de un proceso, aunque el valor esté entre 0 y 19: si el proceso tiene nice 10 y pide 5, `renice` falla y el script lo indica.

```
Opción: 4
PID: 287
Nuevo valor nice: 5
renice: failed to set priority for 287 (process ID): Permission denied
Error: no se ha podido cambiar la prioridad de 287
```